

Traccia Algoritmi - Esercizi 1A e 2A

Provadel22/06/2026 - trascrizione pulita ricavata dalle fotografie fornite. Il PDF contiene il testo degli esercizi, non le soluzioni.

Nota: le fotografie originali sono leggermente inclinate e non perfettamente nitide; la consegna è stata ricostruita mantenendo il contenuto leggibile e la notazione dell'esame.

Esercizio 1 (A) - Albero Binario

Si considerino le seguenti funzioni che implementano un Albero Binario in cui la parte informativa di ogni nodo è un numero intero:

- `createTree()` che restituisce un albero vuoto;
- `insert(A, x)` che inserisce il nodo x nell'albero A ;
- `left(A)` che restituisce il sottoalbero sinistro di A ;
- `right(A)` che restituisce il sottoalbero destro di A ;
- `info(A)` che restituisce il valore associato al nodo radice di A ;
- `setInfo(A, x)` che modifica il valore associato alla radice di A impostandolo a x ;
- `isNull(A)` restituisce `True` se l'albero A è vuoto e `False` altrimenti.

1. Si realizzi una efficiente funzione `pathBalance(A, k)` che, dato un Albero binario A e un intero $k > 0$, restituisce `True` se e solo se esiste in A un cammino $r = v_0, v_1, \dots, v_h = q$ che va dalla radice r di A a una foglia q di A , con $h \geq 1$, tale che valgano entrambe le seguenti proprietà:

- la somma $\text{info}(v_0) + \text{info}(v_1) + \dots + \text{info}(v_h)$ sia divisibile per k (cioè il resto della divisione intera per k sia 0);
- non esistano due nodi consecutivi lungo il cammino (cioè due nodi v_i, v_{i+1}) che abbiano entrambi valore negativo.

2. Si discuta, motivandola adeguatamente, la complessità temporale del metodo nei casi migliore e peggiore, esplicitando quale configurazione costituisca, rispettivamente, il caso migliore e il caso peggiore.

3. Si discuta, motivandola adeguatamente, la complessità spaziale del metodo nei casi migliore e peggiore, esplicitando quale configurazione costituisca, rispettivamente, il caso migliore e il caso peggiore.

4. Si discuta se, assumendo che A sia un Albero Binario di Ricerca, la complessità individuata ai punti precedenti cambierebbe, motivando la risposta.

Esercizio 2 (A) - Grafo non orientato e pesato

Si considerino le seguenti funzioni che implementano un grafo non orientato e pesato:

- `createGraph(m)`, che, dato un valore intero m in input, crea un grafo con m nodi identificati da numeri interi compresi in $[0, m - 1]$, senza archi;
- `insertEdge(G, x, y, w)`, che, dato un grafo G , una coppia di nodi x e y e un numero reale positivo w , aggiunge un arco tra x e y di peso w (se l'arco esiste già, ne aggiorna il peso);
- `deleteEdge(G, x, y)`, che, dato un grafo G e una coppia di nodi x e y , rimuove, se presente, l'arco tra x e y ;
- `deleteNode(G, x)`, che, dato un grafo G e un nodo x , rimuove il nodo x da G . Se il nodo x possiede archi incidenti, oppure x non è presente nel grafo G , il metodo solleva un'eccezione;
- `existsEdge(G, x, y)`, che, dato un grafo G e una coppia di nodi x e y , restituisce `True` se esiste in G un arco tra x e y , `False` altrimenti;
- `getWeight(G, x, y)`, che, dato un grafo G e una coppia di nodi x e y tali che esiste l'arco (x, y) , restituisce il peso di tale arco;
- `size(G)`, che, dato un grafo G , restituisce il numero di nodi del grafo;
- `nodes(G)`, che, dato un grafo G , restituisce una lista contenente gli identificatori dei nodi di G .

Si noti che, a differenza di altre tracce, l'API non fornisce una funzione `neighbors` per ottenere direttamente i vicini di un nodo: tale funzione, se necessaria, dovrà essere implementata a partire dalle primitive sopra elencate.

Una rete di n impianti deve essere collegata tramite cavi, in modo che da ogni impianto sia possibile raggiungere qualsiasi altro impianto. Il grafo G rappresenta tutti i collegamenti tecnicamente realizzabili tra coppie di impianti, dove il peso di un arco è il costo di realizzazione del relativo cavo. Per ragioni tecniche, però, nessun cavo può avere un costo di realizzazione superiore a una soglia massima c data.

1. Si realizzi una funzione `neighbors(G, x)` che, dato un grafo G e un nodo x , restituisca la lista delle coppie $[y, w]$ tali che esiste l'arco (x, y) di peso w , utilizzando esclusivamente le primitive elencate sopra.
2. Si realizzi una funzione `filterGraph(G, c)` che, dato un grafo G non orientato e pesato e un valore reale $c > 0$, restituisca un nuovo grafo G_c , con lo stesso numero di nodi di G , contenente esclusivamente gli archi di G il cui peso è minore o uguale a c (con lo stesso peso dell'arco originale).
3. Si realizzi una funzione `budgetedConnection(G, s, c)` che, dato un grafo G non orientato e pesato, un nodo sorgente s e un valore reale $c > 0$, utilizzi la funzione `filterGraph` del punto precedente e determini un sottoinsieme di archi di G_c che collega tutti i nodi raggiungibili da s in G_c , minimizzando la somma dei pesi degli archi scelti. La funzione deve restituire una coppia (raggiunti, archi), dove raggiunti è l'insieme dei nodi collegati a s nella soluzione costruita e archi è la lista degli archi selezionati, ciascuno rappresentato come tripla $[x, y, w]$. Si noti che, se un nodo y è raggiungibile da s in G solo attraverso cammini che contengono almeno un arco di peso superiore a c , allora y non deve comparire nell'insieme raggiunti.
4. Si analizzi e si motivi la complessità temporale nel caso peggiore di `neighbors`, `filterGraph` e `budgetedConnection`, specificando la struttura dati ausiliaria adottata.
5. Si analizzi e si motivi la complessità spaziale nel caso peggiore di `neighbors`, `filterGraph` e `budgetedConnection`.